PROJECT REPORT

FOR

Foundations of Artificial Intelligence

Smart Hackathon Team Formation System

A PROJECT REPORT

Submitted by

2116240701342 -Mukundhan MV

2116240701087 – Chandru RR



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "**Smart Hackathon Team Formation System using Forward Chaining Algorithm**" is the bonafide work of " Mukundhan MV (240701342) & Chandru RR (240701087) " who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	9
	3.1 ARCHITECTURE DIAGRAM	11
4	MODULES DESCRIPTION	12
	4.1 MODULE 1	12
	4.2 MODULE 2	13
5	IMPLEMENTATION AND RESULTS	15
	5.1 EXPERIMENTAL SETUP	16
	5.2 RESULTS	17
6	CONCLUSION AND FUTURE WORK	20
7	REFERENCES	21

ABSTRACT:

The **Smart Hackathon Team Formation System using Forward Chaining Algorithm** is a web-based platform designed to simplify hackathon management and team formation. The system provides separate functionalities for **students and organizers**, allowing organizers to create hackathons, define rules, manage registrations, and monitor teams.

Students can view published hackathons and register through **Individual, Partial Team, or Full Team** options. Partial teams can discover incomplete teams, send join requests, and merge with other partial teams. The system uses a **Rule-Based Expert System with Forward Chaining** to evaluate facts such as hackathon status, registration type, team capacity, and applicable requirements using predefined **IF–THEN rules**.

The system is developed using **React, TypeScript, Node.js, Express.js, and MongoDB**, with **JWT and bcrypt** for authentication. It provides a structured and explainable approach to team formation while reducing manual effort for students and organizers.

1. INTRODUCTION

Hackathons provide students with opportunities to solve real-world problems, develop innovative solutions, and collaborate with other participants. However, forming a suitable team within a limited time can be challenging. Students may struggle to find incomplete teams, join existing teams, or combine partial teams to satisfy the required team size. Organizers also face difficulties in managing hackathons, defining requirements, handling registrations, and monitoring team formation.

The **Smart Hackathon Team Formation System using Forward Chaining Algorithm** is a web-based platform developed to simplify hackathon management and team formation. The system provides separate functionalities for **students and organizers**. Organizers can create hackathons, define rules and requirements, publish or close hackathons, and manage registrations and teams. Students can view published hackathons and register through **Individual, Partial Team, or Full Team** options.

The system uses a **Rule-Based Expert System with Forward Chaining** as its AI approach.. Predefined **IF–THEN rules** are then applied to these facts to derive decisions. This allows the system to determine whether a student can register, join an incomplete team, or merge a partial team.

1. The system also provides **join requests, merge requests, acceptance/rejection mechanisms, and notifications** to support the complete team formation process. When students select the "Find a Team" option, the system displays incomplete teams for the selected hackathon without filtering them based on skills, department, gender, year, or compatibility.
2. The system is developed using **React, TypeScript, and Tailwind CSS** for the frontend, **Node.js and Express.js** for the backend, and **MongoDB** for database management. **JWT and bcrypt** are used for secure authentication. The proposed system reduces

manual effort and provides a centralized, structured, and explainable approach to hackathon team formation.

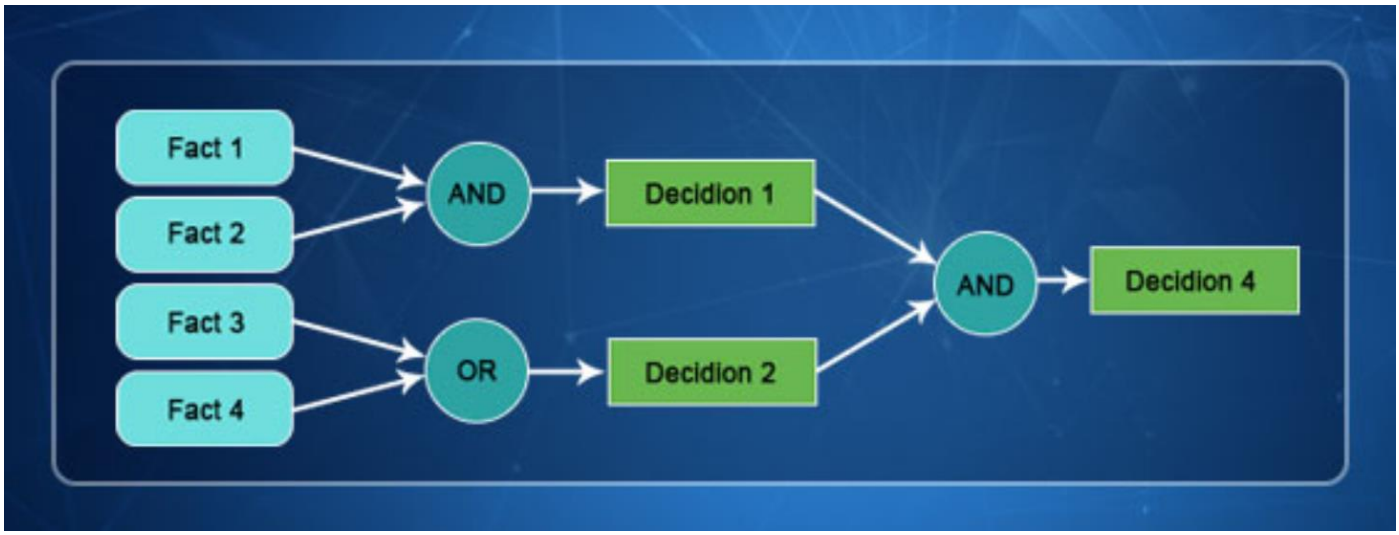


Fig 1.1 Forward Chaining Algorithm

2. LITERATURE REVIEW

Rule-Based Expert Systems for Team Formation

Stavrou et al. (2023) studied team formation through a systematic literature review and highlighted that team formation involves multiple factors such as member characteristics, task requirements, preferences, and team objectives. Their work demonstrated the importance of structured approaches for forming effective teams. This concept influenced the development of the **rule-based team formation approach** in the proposed Smart Hackathon Team Formation System.

Donndelinger et al. (2021) investigated automated project team formation by considering project preferences and skill-mix constraints. Their research demonstrated how predefined requirements can be used to support automated team formation and ensure that teams satisfy specified conditions. This concept is relevant to the proposed system, where hackathon requirements and team-size conditions are evaluated using predefined rules.

Forward Chaining and Knowledge-Based Reasoning

Lappas, Liu, and Terzi (2009) studied the problem of finding teams of experts in social networks and considered task requirements and relationships between potential team members. Their work demonstrated how computational reasoning can support team formation based on predefined conditions. This provides a foundation for applying structured reasoning to team formation problems.

Murphy et al. (2022) investigated student team formation by considering factors such as soft skills and common availability. Their study demonstrated that practical constraints can be incorporated into automated student team formation. This concept relates to the proposed system, where predefined hackathon and team requirements are used to validate team formation.

Hackathon Team Formation

Nalawade et al. (2025) examined smart team-building approaches for college hackathons and discussed factors such as skills, experience, interests, and participant characteristics. Their work highlights the growing application of intelligent approaches to hackathon team formation. However, such approaches primarily focus on recommending suitable team members.

The proposed **Smart Hackathon Team Formation System** focuses on the complete team-formation workflow, including **Individual, Partial Team, and Full Team registration, incomplete-team discovery, join requests, merge requests, and rule-based validation**. The system uses **Forward Chaining** to process known facts through predefined IF–THEN rules and derive decisions for registration, joining, and team merging. Future enhancement can incorporate AI-based skill and compatibility matching.

3. PROPOSED SYSTEM

The proposed **Smart Hackathon Team Formation System using Forward Chaining Algorithm** is a web-based platform designed to simplify hackathon management and student team formation. The system uses a **Rule-Based Expert System with Forward Chaining** to validate registration and team formation decisions.

The system provides two roles: **Student and Organizer**. Organizers can create hackathons, define rules and requirements, publish or close hackathons, and manage registrations and teams. Students can view published hackathons and register using **Individual, Partial Team, or Full Team** options.

The **Forward Chaining engine** starts with known facts such as hackathon status, registration type, team capacity, student eligibility, and existing team conditions. It applies predefined **IF–THEN rules** to derive decisions for registration, team formation, joining, and merging.

Students with partial teams can view **incomplete teams** for the selected hackathon and send join requests. Partial teams can also send merge requests to other partial teams. Team leaders can accept or reject these requests, and notifications are provided for request updates.

Major Components

1. **Authentication Module**
    - Student and organizer registration/login.
    - JWT authentication and bcrypt password security.
2. **Hackathon Management Module**
    - Create and edit hackathons.
    - Define rules and requirements.
    - Publish and close hackathons.

3. **Registration Module**
    - Individual registration.
    - Partial Team registration.
    - Full Team registration.
    - Registration validation.
4. **Team Management Module**
    - Create and manage teams.
    - Manage team members.
    - Maintain team size and status.
5. **Forward Chaining Module**
    - Collects system facts.
    - Applies predefined IF–THEN rules.
    - Determines whether registration, joining, or merging can proceed.
6. **Join and Merge Request Module**
    - View incomplete teams.
    - Send join requests.
    - Send merge requests between partial teams.
    - Accept or reject requests.
7. **Notification Module**
    - Provides notifications for join and merge request activities.

3.1 ARCHITECTURE DIAGRAM

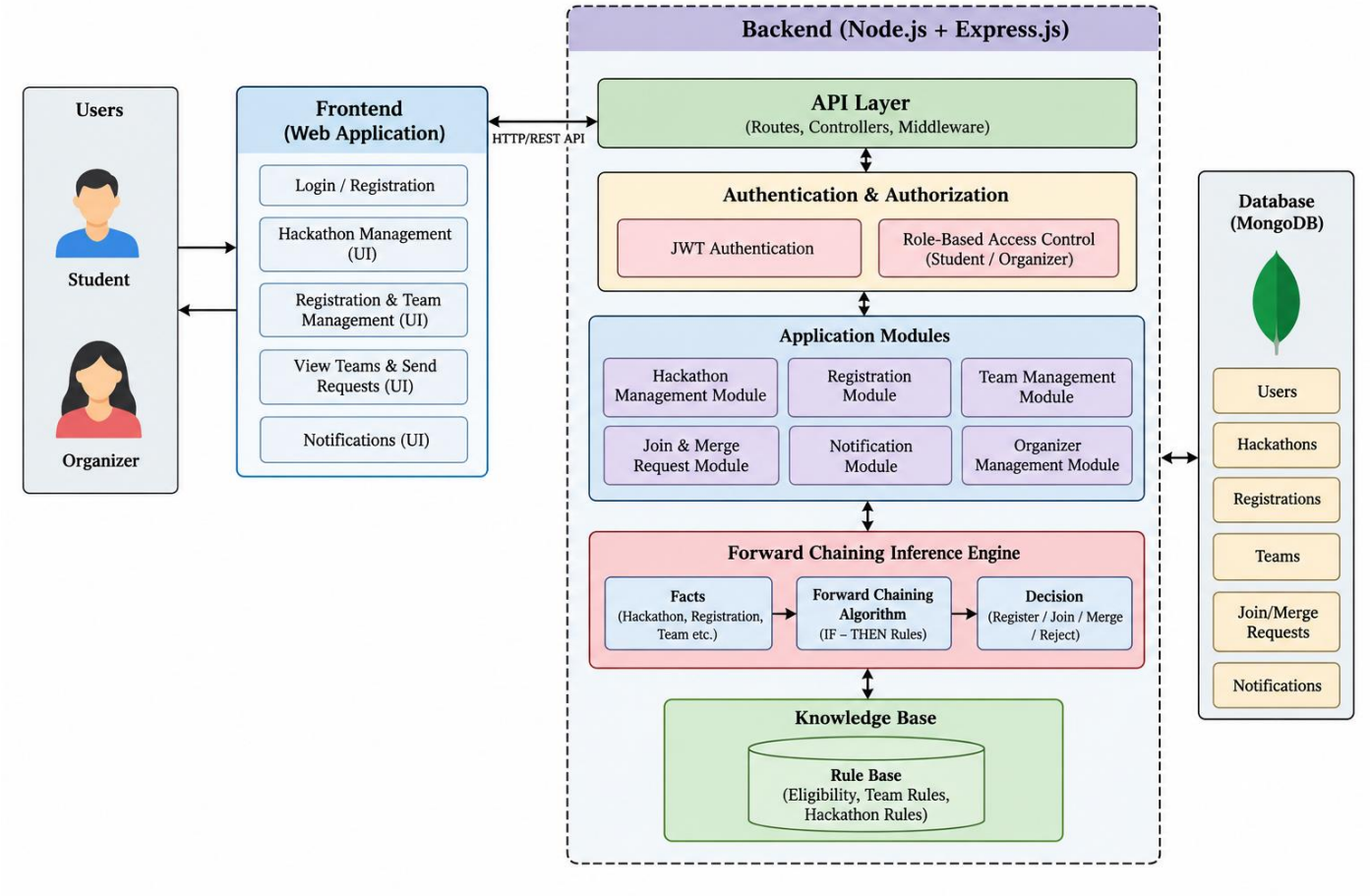


Figure 3.1: System Architecture

4. MODULES DESCRIPTION

The Smart Hackathon Team Formation System is organized into two major modules that work together to provide a structured hackathon management and team formation environment. The first module manages user interaction, authentication, hackathon management, registration, and team-related activities, while the second module manages the Forward Chaining inference, rule processing, join and merge validation, and backend operations. The integration of these modules allows students to register for hackathons, form teams, find incomplete teams, send requests, and complete team formation, while organizers can manage hackathons, rules, registrations, and teams.

4.1 MODULE 1 – USER INTERFACE AND HACKATHON MANAGEMENT MODULE

The User Interface and Hackathon Management Module forms the primary interaction layer of the system. It provides a web interface where students and organizers can register, log in, view hackathons, manage hackathon information, register for hackathons, and participate in team formation. The module is developed using React.js, TypeScript, and Tailwind CSS and communicates with the backend through REST APIs.

The module provides separate interfaces for students and organizers. Organizers can create hackathons, define rules and requirements, edit hackathon details, publish or close hackathons, and manage registrations and teams. Students can view published hackathons, view hackathon details and requirements, and select Individual, Partial Team, or Full Team registration.

The module also provides interfaces for creating and managing teams, viewing incomplete teams, sending join requests, sending merge requests, and receiving notifications related to team activities.

Main functions:

- Student and organizer registration and login.
- JWT-based authentication.
- Hackathon creation and editing.
- Definition of hackathon rules and requirements.
- Publishing and closing hackathons.
- Viewing published hackathons.
- Individual, Partial Team, and Full Team registration.
- Team creation and management.
- Viewing incomplete and partial teams.
- Sending join and merge requests.
- Viewing notifications.

4.2 MODULE 2 – TEAM FORMATION AND FORWARD CHAINING MODULE

The Team Formation and Forward Chaining Module is responsible for processing team formation activities and applying the predefined rules of the system. The backend is developed using Node.js, Express.js, and TypeScript and communicates with the frontend through REST APIs.

The module collects facts such as hackathon status, registration type, student registration status, team size, and applicable hackathon requirements. These facts are processed using a Rule-Based Expert System with Forward Chaining.

The Forward Chaining engine evaluates the available facts against predefined IF–THEN rules. When the conditions of a rule are satisfied, the corresponding conclusion is derived. The process continues until the required decision is reached.

The Forward Chaining process supports decisions related to:

- Registration validation.
- Team formation.
- Join requests.
- Merge requests.
- Team-size validation.
- Hackathon requirement validation.
-

The system also allows students to view incomplete teams for the selected hackathon. Team discovery is not filtered by skills, department, year, gender, interests, or compatibility. Applicable hackathon requirements are used for validation where required.

Main functions:

- REST API request handling.
- JWT authentication and authorization.
- Student and organizer validation.
- Hackathon and registration validation.
- Fact collection and rule processing.
- Forward Chaining inference.
- Team-size validation.
- Join request processing.
- Merge request processing.
- Accepting and rejecting requests.
- Team member management.
- Notification generation.

5. IMPLEMENTATION AND RESULTS

The Smart Hackathon Team Formation System was implemented as a web-based application that combines a modern hackathon management interface with a rule-based team formation system. The frontend was developed using React.js, TypeScript, and Tailwind CSS, while the backend was implemented using Node.js, Express.js, and TypeScript. MongoDB was used for storing users, hackathons, registrations, teams, requests, and notifications.

The application connects the frontend and backend through REST API communication. JWT authentication and bcrypt are used to provide secure authentication and role-based access for students and organizers. Organizers can create and manage hackathons, while students can view published hackathons and register using Individual, Partial Team, or Full Team options.

The Forward Chaining engine evaluates the available facts against predefined knowledge-base rules. Based on the satisfied conditions, the system determines whether registration, team formation, joining, or merging can proceed. The system also manages join and merge requests and updates team information based on the request outcomes.

The implemented system provides a structured team formation process through registration, incomplete-team discovery, join requests, merge requests, and notifications. The rule-based approach ensures that team formation activities follow the predefined hackathon and team requirements.

5.1 EXPERIMENTAL SETUP

The frontend of the Smart Hackathon Team Formation System was developed using React.js, TypeScript, and Tailwind CSS to provide the user interface for students and organizers. The frontend communicates with the backend through REST APIs.

The backend was developed using Node.js, Express.js, and TypeScript. JWT is used for authentication, while bcrypt is used for secure password handling. MongoDB is used to store user accounts, hackathons, registrations, teams, join requests, merge requests, and notifications.

The AI component consists of a Rule-Based Expert System using the Forward Chaining inference algorithm. The system evaluates predefined IF–THEN rules using facts such as hackathon status, registration type, team capacity, student eligibility, and team conditions.
The experimental setup includes different team formation scenarios such as:

- Student registers individually → Individual registration.
- Student creates an incomplete team → Partial Team formation.
- Complete team registers → Full Team registration.
- Student selects an incomplete team → Join request is processed.
- Two partial teams satisfy the team-size condition → Merge request is processed.
- Required conditions are not satisfied → Registration or team request is rejected.

The system verifies these scenarios to ensure that registration, team formation, joining, and merging follow the predefined rules and team-size requirements.

5.2 RESULT

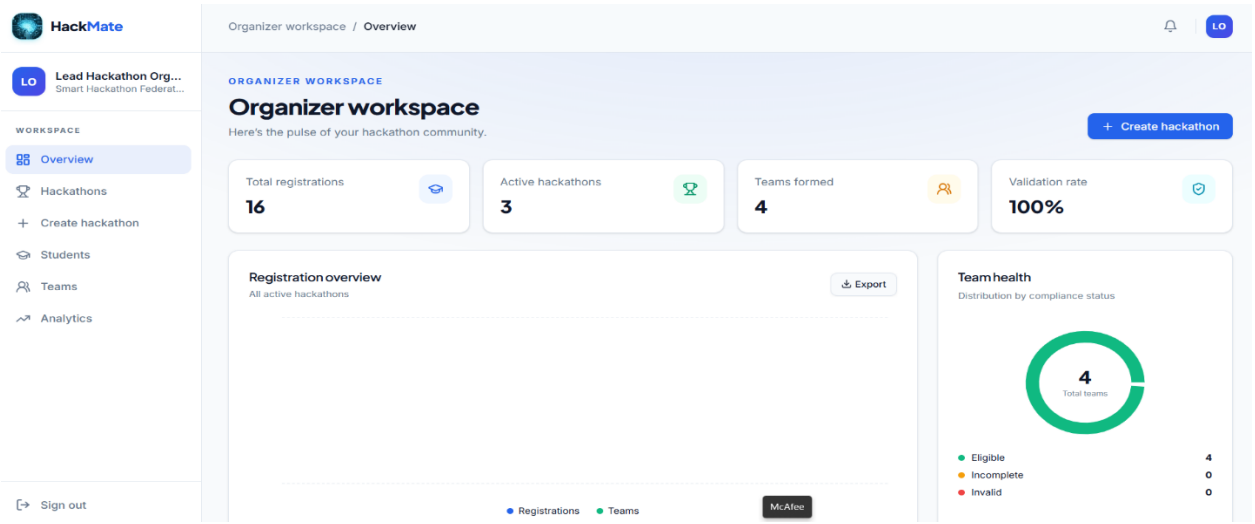


Figure 5.2.1 Organiser Overview

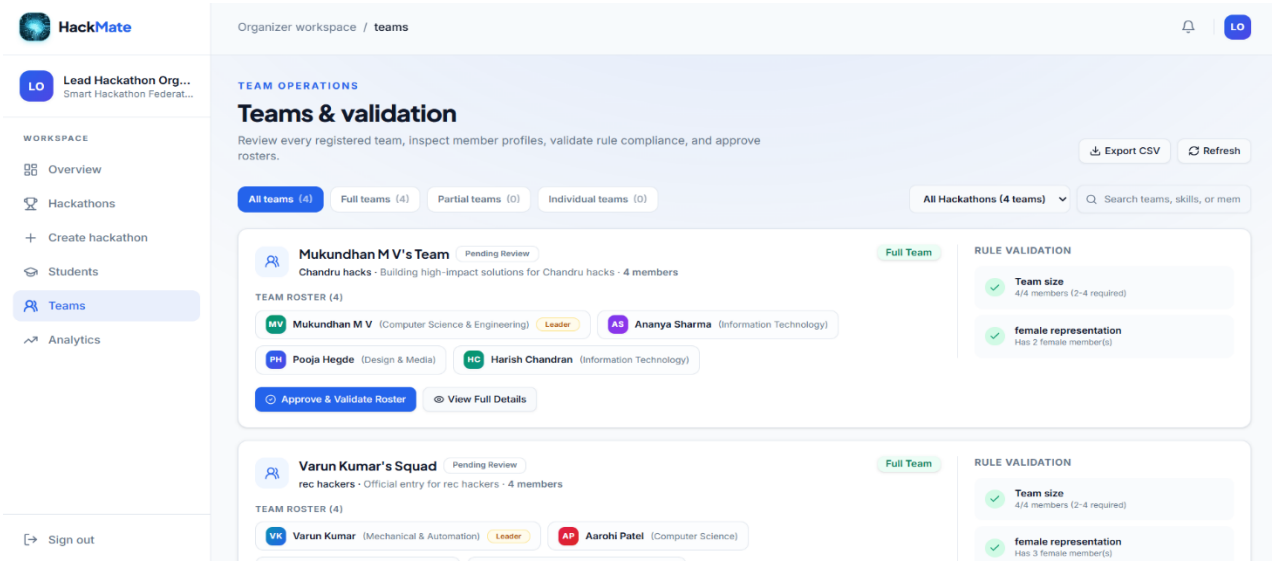


Figure 5.2.3 Organiser Team Validation page

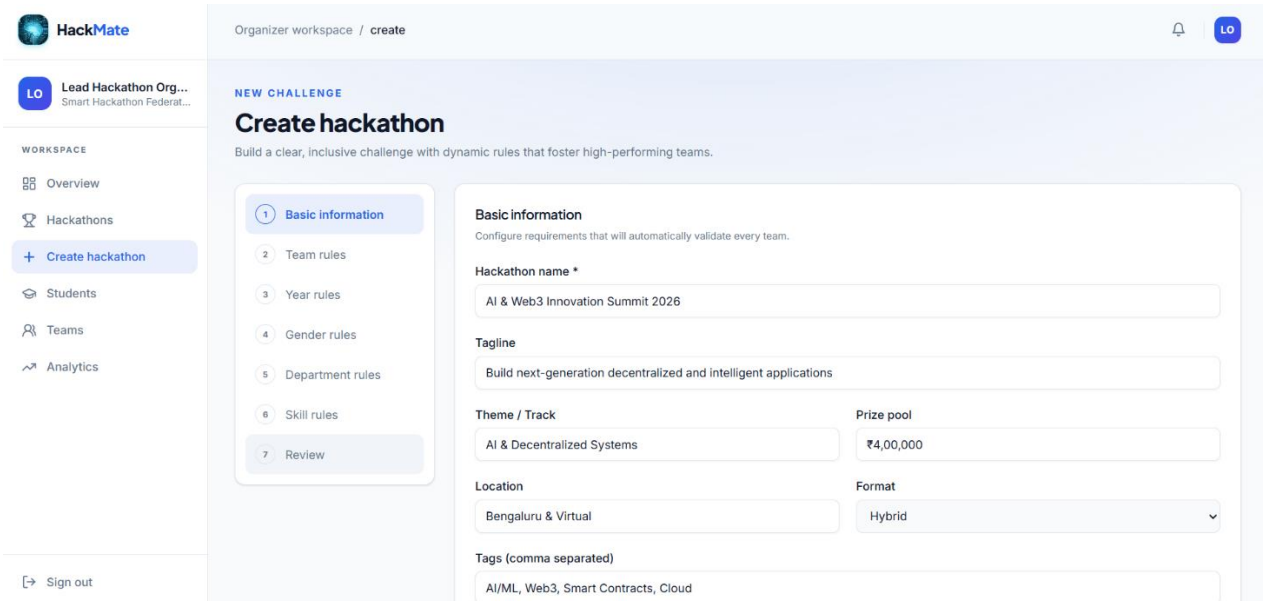


Figure 5.2.3 Hackathon creation page

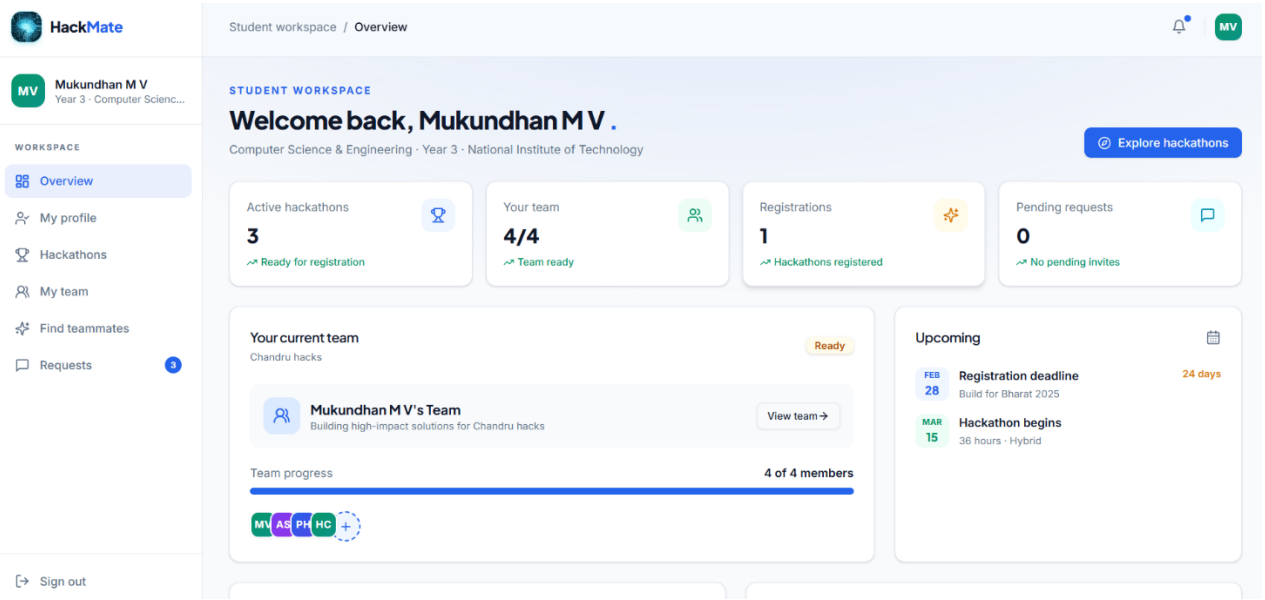


Figure 5.2.4 Student Overview

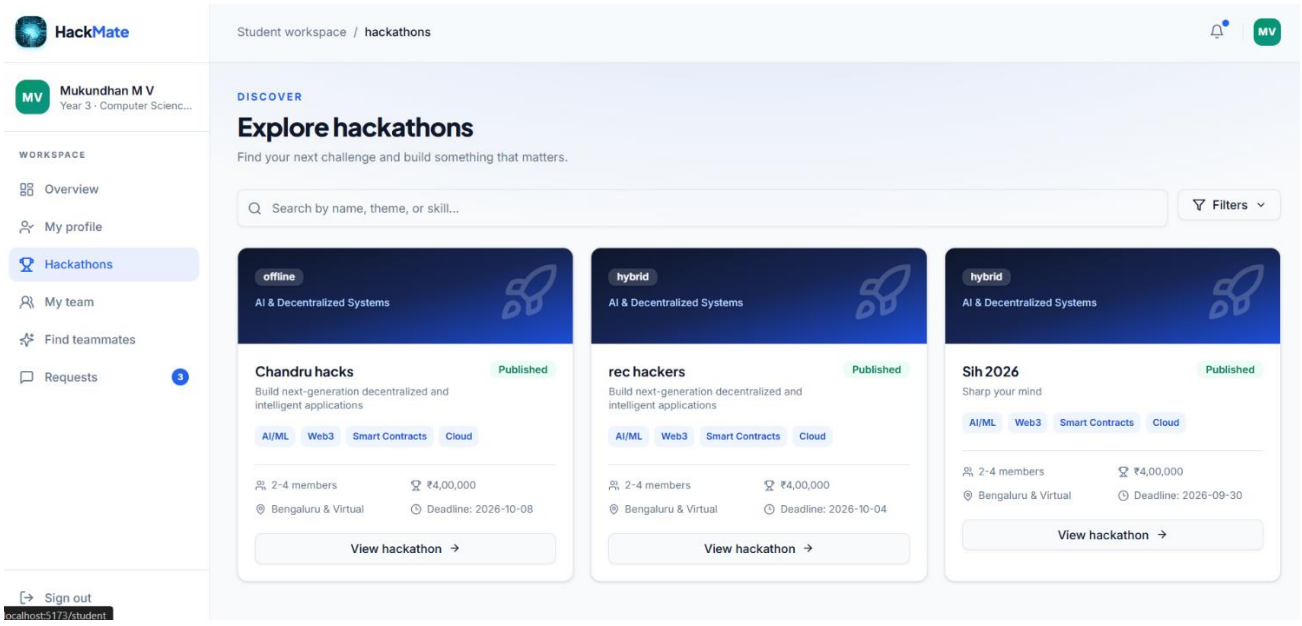


Fig 5.2.5 Available Hackathons

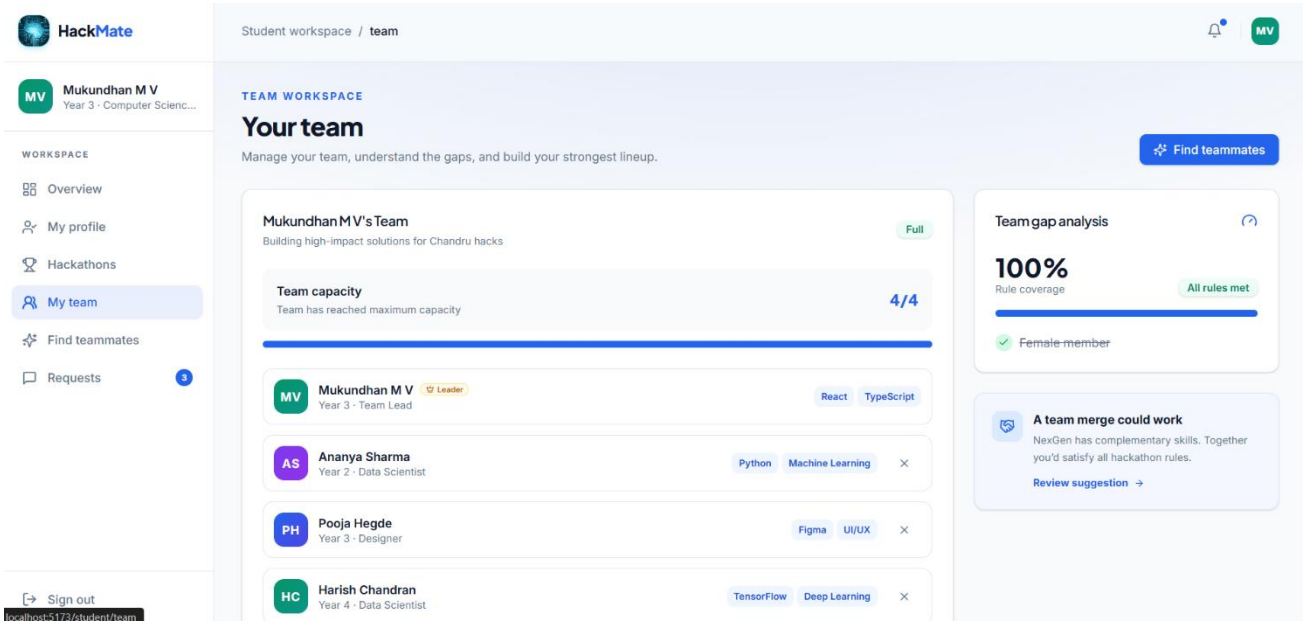


Figure 5.2.7 Student Team

5. CONCLUSION AND FUTURE WORK

The Smart Hackathon Team Formation System using Forward Chaining Algorithm is a rule-based platform developed to simplify hackathon management and student team formation. The system integrates React.js, TypeScript, Node.js, Express.js, MongoDB, JWT authentication, a Rule-Based Expert System, and a Forward Chaining inference mechanism.

The system supports Individual, Partial Team, and Full Team registration along with incomplete-team discovery, join requests, merge requests, request acceptance or rejection, and notifications. The Forward Chaining engine evaluates predefined IF–THEN rules based on facts such as hackathon status, registration type, team capacity, and applicable requirements. This approach makes the team formation process structured, consistent, and explainable.

The implemented system provides a centralized environment for organizers to manage hackathons and for students to form teams efficiently. The rules used for team formation are designed for the academic implementation of the project.

FUTURE WORK

The system can be enhanced in the future by introducing AI-based teammate recommendation, skill and interest-based matching, and compatibility analysis between students. These enhancements can help students identify suitable teammates based on their skills and interests.

Future improvements can also include advanced analytics for organizers and mobile application support. These features can improve hackathon **management and make the team formation system more accessible and efficient.**

REFERENCES

1. Stavrou, E., et al. (2023) 'Team Formation: A Systematic Literature Review', *International Journal of Human-Computer Studies*.
2. Donndelinger, E., et al. (2021) 'Automated Project Team Formation Using Preferences and Skill-Mix Constraints', *Proceedings of the International Conference on Software Engineering and Knowledge Engineering*.
3. Lappas, T., Liu, K. and Terzi, E. (2009) 'Finding Experts in Social Networks', *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1059–1068.
4. Murphy, C., et al. (2022) 'Automated Student Team Formation Using Soft Skills and Availability', *IEEE International Conference on Teaching, Assessment and Learning for Engineering*.
5. Nalawade, S., et al. (2025) 'Smart Team Building for College Hackathons', *International Conference on Emerging Technologies and Applications*.
6. Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*. 4th edn. Pearson.
7. Python Software Foundation (2025) 'Python Documentation', Python Software Foundation, Available at: Python Documentation.
8. Meta Platforms, Inc. (2025) 'React Documentation', Meta Open Source, Available at: React Documentation.
9. MongoDB, Inc. (2025) 'MongoDB Documentation', MongoDB, Available at: MongoDB Documentation.
10. Express.js (2025) 'Express.js Documentation', Express.js, Available at: Express.js Documentation.
11. OpenJS Foundation (2025) 'Node.js Documentation', Node.js, Available at: Node.js Documentation.
12. GitHub, Inc. (2025) 'GitHub Documentation', GitHub, Available at: GitHub Documentation.